

One Model, Many Roles

Какие внутренние представления LLM сохраняются между агентскими ролями?

1 Проблема и основной вопрос

Одна и та же frozen LLM в агентной системе может выступать как **Solver, Planner, Executor, Critic, Verifier** или **Tool User**. Веса модели остаются одинаковыми, но меняются контекст, цель текущего вызова и тип вычисления, которое модель должна выполнить.

В интерпретируемости часто показывают, что некоторый концепт можно декодировать из hidden states: корректность, уверенность, необходимость tool call и т.д. Однако почти всегда probe обучается и тестируется в одном режиме работы модели.

Главный вопрос работы:

Какие внутренние переменные одной и той же LLM имеют общий, role-invariant код, а какие перекодируются или исчезают при смене agentic role?

Нас интересует не только ошибка. Мы хотим понять, **какие элементы внутреннего “состояния агента” вообще универсальны для разных способов использования одной модели.**

2 Что именно мы хотим пробить

В основной работе разумно исследовать четыре группы сигналов.

1. Correctness / failure — простой baseline

Можно ли по текущему hidden state предсказать, закончится ли trajectory правильным ответом?

$$y_t^{\text{success}} = \mathbb{I}[\text{trajectory eventually succeeds}].$$

Это самый простой target с автоматическим ground truth, но он не является основной идеей статьи.

2. Epistemic state — что модель знает и чего ей не хватает

Здесь наиболее интересны:

- **uncertainty**: насколько текущее состояние ведёт к стабильному правильному решению;
- **evidence sufficiency**: достаточно ли уже информации для ответа или необходимо продолжать поиск;
- **belief**: какую из конкурирующих гипотез модель сейчас внутренне поддерживает.

Например, uncertainty можно определять не по словам модели, а эмпирически: из одного и того же состояния запускать K continuations и использовать

$$u_t = 1 - \frac{1}{K} \sum_{k=1}^K \mathbb{I}[\text{continuation}_k \text{ succeeds}].$$

Так мы сравниваем verbal confidence с реально присутствующим latent signal.

3. Computational state — что модель считает нужным делать

Здесь исследуем:

- **current subgoal**: какой подшаг задачи сейчас активен;
- **progress**: насколько далеко модель продвинулась в решении;
- **next useful computation**: что сейчас полезнее — reason, search, code, verify или stop;
- **tool necessity / affordance**: нужен ли инструмент и какого типа.

Это особенно важно для agentic setting: один и тот же semantic state может обрабатываться Planner’ом, Executor’ом или Critic’ом, и мы проверяем, существует ли общий внутренний код “что делать дальше”.

4. Control / trustworthy-AI signals

Дополнительный блок:

- осознаёт ли модель конфликт инструкций;
- различает ли authority системной инструкции, user message и tool output;
- понимает ли границы своей роли и ответственности;
- распознаёт ли небезопасное или неподтверждённое действие до генерации.

Для первой версии статьи достаточно трёх основных targets:

success + evidence/uncertainty + next useful computation
--

Correctness даёт простой baseline, а два последних target’а показывают, есть ли более общий role-invariant “agent state”.

3 Агентские сетапы

Сначала используем одну frozen open-weight модель (например, Qwen3-8B/14B), но помещаем её в разные роли.

1. **Solver**: самостоятельно решает задачу целиком.
2. **Planner**: строит последовательность subgoals, но не исполняет её.
3. **Executor**: получает готовый subgoal и выполняет его.
4. **Critic**: анализирует предложенное решение и ищет ошибку.
5. **Verifier**: принимает решение accept/reject по answer/evidence.
6. **Tool User**: выбирает Search / Python / Browser / Stop.

Базовые workflows:

Solver,

Planner $\rightarrow$ Executor,

Solver $\rightarrow$ Critic $\rightarrow$ Revision,

Planner $\rightarrow$ Executor $\rightarrow$ Verifier,

а затем ReAct/tool-use и более сложные MAS.

Ключевой принцип: **weights модели фиксированы**. Меняем только вычислительную роль и контекст.

4 Что извлекаем из модели

Пусть

$$h_{r,t}^{(\ell)} \in \mathbb{R}^d$$

— hidden state слоя ℓ на шаге t в роли r .

Не выбираем один слой заранее. Сохраняем activations всех transformer blocks и строим **layer-wise profile**.

Основные точки чтения:

- последний token перед генерацией ответа;
- состояние непосредственно перед выбором следующего action/tool;
- при необходимости — max/mean pooling по reasoning span.

Это важно: свежие работы показывают, что универсальность внутренних сигналов может сильно зависеть от слоя и способа pooling.

5 Основная математика

Для каждого concept c , роли r и слоя ℓ обучаем простой linear probe:

$$p(c = 1 \mid h) = \sigma \left((w_{r,c}^{(\ell)})^\top h + b_{r,c}^{(\ell)} \right).$$

Главный эксперимент — **cross-role transfer**:

$$A_{r \rightarrow s, c}^{(\ell)} = \text{AUROC} \left(P_{r,c}^{(\ell)} \text{ tested on role } s \right).$$

Получаем transfer matrix для каждого concept и слоя.

Дальше сравниваем три режима.

Universal code

$$A_{r \rightarrow s, c}^{(\ell)} \approx A_{s \rightarrow s, c}^{(\ell)}.$$

Probe переносится: concept кодируется сходным образом в разных ролях.

Role-specific recoding

$$A_{r \rightarrow s, c}^{(\ell)} \ll A_{s \rightarrow s, c}^{(\ell)},$$

но новый probe на роли s работает хорошо.

Информация есть в обеих ролях, но лежит в разных directions/subspaces.

Loss of monitorable information

Даже role-specific probe работает плохо.

Значит данная вычислительная роль сама по себе слабее представляет интересующий concept.

6 Как отделить настоящий эффект от prompt shift

Это критическая часть работы. Иначе любой результат можно объяснить тем, что prompts просто разные.

Нужны matched controls:

- **prompt paraphrase**: разные формулировки внутри одной роли;
- **role-label control**: одинаковый semantic input, но разная role instruction;
- **matched information**: роли получают максимально одинаковые факты/evidence;
- **context-length control**: отдельно контролируем длину history;
- **format deconfounding**: labels не должны совпадать с очевидными шаблонами текста или формата.

Главный claim должен быть не “роль меняет hidden states” — это очевидно, а:

после контроля surface-level различий некоторые computational variables сохраняют общий latent code между ролями, а другие систематически становятся role-specific.

7 Дополнительный mechanistic анализ

Если cross-role effect найден, исследуем его геометрию:

- cosine similarity между probe directions;
- principal angles между concept subspaces;
- СКА / representation similarity;
- PCA для наглядной визуализации;
- linear alignment / Procrustes mapping между ролями.

Далее — causal validation. Например, steering:

$$\tilde{h}^{(\ell)} = h^{(\ell)} + \alpha w_c^{(\ell)}.$$

Особенно интересный тест: берём direction, найденный у Solver, и вмешиваемся вдоль него в Planner или Executor.

Если probe переносится, но causal effect не переносится, то

shared decodable representation $\neq$ shared computational mechanism

— это потенциально сильный результат статьи.

8 Зачем это нужно

Практический trustworthy-AI вопрос:

Можно ли обучить внутренний monitor на одной deployment role и затем доверять ему, когда ту же модель используют как Planner, Executor, Critic или Tool User?

Более фундаментальный вопрос:

Существует ли у LLM общий внутренний “agent state” — belief, uncertainty, progress, action affordance — или разные agentic computations используют разные внутренние коды?

Если сигналы универсальны, можно строить переносимые monitors. Если они role-specific, monitoring необходимо валидировать для конкретного способа использования модели. Если некоторые роли делают важную информацию менее decodable, архитектура агента влияет не только на capability, но и на observability.

9 Минимальный пилот

Первый эксперимент должен быть дешёвым:

$$1 \text{ model} \times 2 \text{ roles} \times \sim 1000 \text{ tasks} \times \text{all layers}.$$

Начать с:

Solver vs Critic

и трёх сигналов:

success, evidence sufficiency, next useful action.

Если cross-role transfer отличается от within-role performance, добавляем Planner/Executor и второй model family. Если transfer почти идеален, это тоже содержательный результат: может существовать действительно универсальный latent code.

10 Похожие работы и отличие

Точных работ с постановкой “одна frozen LLM $\times$ разные computational agent roles $\times$ systematic cross-role transfer нескольких внутренних agent-state variables” в найденной литературе нет. Но есть несколько близких направлений, от которых нужно явно отстроиться.

- **Poulis et al., 2026, Testing the Limits of Truth Directions in LLMs.** Показывают, что truth/correctness directions зависят от слоя, task type и инструкции. Это ближайшая работа по вопросу универсальности latent directions, но она не изучает agentic roles и agent-state variables.
- **Sun et al., 2026, LLM Agents Already Know When to Call Tools.** Показывают, что необходимость tool call линейно декодируется до генерации. Это подтверждает, что action-affordance signal имеет смысл пробить, но работа не исследует его перенос между Planner/Executor/Critic.

- Ustaomeroglu et al., ICLR 2026, **Internal Planning in Language Models**. Изучают planning horizon и branch awareness в hidden states. Фокус — внутреннее планирование standalone LM, а не сравнение одного внутреннего состояния между разными agentic computations.
- Chrabaszcz et al., 2026, **Monitoring the Internal Monologue**. Строят temporal probe trajectories для прогнозирования будущего поведения. Это близко по monitoring, но не исследует role invariance.
- Qin et al., 2026, **The Granularity Axis**. Показывают структурированный и причинно значимый latent axis для prompted social roles. Это подтверждает, что role conditioning имеет измеримую геометрию, но социальные personas существенно отличаются от computational roles Planner/Executor/Critic.
- Ye et al., 2026, **Prompt Injection as Role Confusion**. Пробуют внутреннее восприятие System/User/Tool authority. Это близко к trustworthy-AI части, но исследует provenance/authority, а не инвариантность computational state между agent roles.
- Fomin et al., 2026, **Internal-State Probes Read the Situation, Not the Action**. Показывают, что хороший probe может считывать scenario/context, а не будущий action, и требуют строгих generalization controls. Эта работа особенно важна методологически: наши matched controls должны исключить именно такие confounds.

Таким образом, novelty работы должна быть сформулирована не как “мы нашли ещё несколько линейно декодируемых концептов”, а как:

систематическое исследование того, какие внутренние computational/epistemic variables одной frozen LLM являются role-invariant в agentic systems, а какие зависят от выполняемой роли, с контролем confounds и causal validation.

11 Ссылки

- Poulis et al., *Testing the Limits of Truth Directions in LLMs*:
<https://arxiv.org/abs/2604.03754>
- Sun et al., *LLM Agents Already Know When to Call Tools*:
<https://arxiv.org/abs/2605.09252>
- Ustaomeroglu et al., *Internal Planning in Language Models*:
https://proceedings.iclr.cc/paper_files/paper/2026/hash/da004051ce3fb70898f12395fc0c1fc3-Abst.html
- Chrabaszcz et al., *Monitoring the Internal Monologue*:
<https://arxiv.org/abs/2605.18549>
- Qin et al., *The Granularity Axis*:
<https://arxiv.org/abs/2605.06196>
- Ye et al., *Prompt Injection as Role Confusion*:
<https://arxiv.org/abs/2603.12277>
- Fomin et al., *Internal-State Probes Read the Situation, Not the Action*:
<https://arxiv.org/abs/2606.30449>